

FastGRNN-HAR v2 — Türkçe Taslak (çalışma dosyası)

Workflow: önce full Türkçe taslak, sonra topluca İngilizce'ye çevirip `paper/en/sections/*.tex` 'e işlenecek (mevcut dosyalar v1, üzerine yazılacak). Framing kilidi: bkz. `memory project-fastgrnn-scope-stance`.

Durum (31 Tem 2026): Abstract + Giriş + İlgili Çalışma onaylı. Sıra: Yöntem.

Abstract (onaylı TR)

Ultra-kısıtlı, çarpıcısız bir mikrodenetleyicide gerçek-zamanlı sinirsel çıkarım, modele mi bağlıdır yoksa nasıl dağıtıldığına mı? Bu soruyu, bir TI MSP430G2553 üzerinde — 16 KB Flash, 512 B SRAM, donanım çarpıcısı yok, FPU yok — insan aktivite tanıma (HAR) örneğinde, iddiayla değil ölçümle yanıtıyoruz. Üç kompakt yinelemeli hücreyi (GRU, LSTM, FastGRNN) aynı dağıtım mühendisliği altında değerlendirdiğimizde, fizibilitenin hücre seçiminden çok iki bağımsız mühendislik ön-koşulu tarafından belirlendiğini buluyoruz: arama-tablosu (LUT) aktivasyonları ve yeterince hızlı (100 kHz) bir sensör veriyolu. Gerçek MPU6050 ivmeölçer döngüdeyken, üç hücre de 50 Hz gerçek-zamanlı çıkarımı (uçtan uca 12–14 ms) ancak ikisi de sağlandığında sürdürebiliyor; birini kaldırınca — yazılımsal transandantal fonksiyonlar ya da tek başına 8.4 ms süren okumasıyla bütçeyi aşan 10 kHz'lik bir veriyolu — her konfigürasyon gerçek-zamanı kaçııyor. Dolayısıyla sensör-okuma yolu birinci derecede belirleyicidir ve çıkarım-only kıyaslamalara görünmezdir. Hangi hücrenin "en iyi" olduğu ise rejime bağlıdır: eşit gizli boyutta GRU her veri setinde önde; eşit *bayt*-bütçesinde ise FastGRNN'in yapısal sıkıştırması sıralamayı verinin desteklediği yerde tersine çeviriyor (WISDM, +0.067 makro-F1, bootstrap-anlamlı; HAPT'te istatistiksel beraberlik). Gecikme, enerji, Flash ve SRAM'i ölçülmüş olarak raporluyoruz (enerji için INA226), FPU'suz bir parçada derleyici optimizasyonunun neden neredeyse işe yaramadığını mekanizmasıyla açıklıyoruz ve üç mimari sınıfı boyunca tahmin düzeyinde çapraz-platform özdeşliğini doğruluyoruz — 8-bit AVR, 16-bit MSP430 ve 32-bit ARM Cortex-M0+ — burada bir donanım çarpıcısının, aktivasyon tablosuyla kabaca aynı gerçek-zaman payını geri kazandığını gösteriyoruz. Katkımız kazanan bir hücre değil; yeterli dağıtım mühendisliğiyle, üretimdeki en küçük silikonların bir kısmında gerçek-zamanlı yinelemeli algılamanın mümkün olduğunun ölçülmüş bir gösterimi — ve tam olarak ne zaman mümkün olmadığının karakterizasyonu.

1. Giriş (onaylı TR)

Motivasyon

Modern makine öğrenmesinin baskın yönü ölçeklenmek oldu: daha büyük modeller, daha büyük hızlandırıcılar, daha büyük bellek bütçeleri. Bu yönün maliyetleri artık makro ölçekte görünür — 2020–2024 arası küresel yarı iletken darboğazı [cite] ve sürekli-çevrimiçi çıkarımın büyüyen enerji/karbon yükü [cite]. Buna karşılık, zaten yılda on milyarlarca adet üretilen, tek haneli dolar maliyetli 8- ve 16-bit mikrodenetleyiciler — giyilebilirlerin, sensörlerin, endüstriyel uç noktaların silikonu — gerçek-zamanlı sinirsel algılama için doğal ama görece az kullanılmış bir zemin sunar. Bunların hiçbirisi yeni bir gözlem değildir; tinyML çalışmaları [cite] tam da bu zemini hedefler.

Daha az ele alınan soru şudur: böyle bir donanımda bir modelin gerçek-zamanlı çalışabilir *olup olmadığını* önceden nasıl bilebiliriz? Modelleri çoğunlukla soyutlamalarla değerlendiririz — bir test kümesindeki doğruluk, parametre sayısı, sensörden yalıtılmış çıkarım gecikmesi. Bu soyutlamalar, gerçek bir dağıtımda fizibiliteyi belirleyen etkenleri dışarıda bırakır: aktivasyonun nasıl hesaplandığı, sensör verisinin çevrim içine nasıl girdiği, altta yatan aritmetik zemin ve hangi kaynak kısıtının bağlayıcı olduğu. Bu boşluğun varlığı bilinmez değildir; ancak gerçek donanımda, sensör döngü içindeyken ve ölçülmüş enerjiyle *ne kadar büyük* olduğu nadiren nicelleştirilir. Bu çalışma onu, boşluğun en genişlediği yerde — çarpıcısı ve kayan-nokta birimi olmayan, sub-kilobayt bir mikrodenetleyicide — ölçer.

Köprü

Örnek problem olarak insan aktivite tanımayı (HAR) alıyoruz — kol ya da bele takılı ataletsel bir sensörden okunan, düşük boyutlu, düşük örnekleme hızlı, kanonik bir akış-sınıflandırma görevi. Ancak makalenin nesnesi HAR değil, dağıtımın kendisidir; HAR yalnızca onu somutlaştıran araçtır. Üç kompakt yinelemeli hücreyi (GRU, LSTM, FastGRNN) hiçbirini savunmadan, aynı dağıtım mühendisliği altında karşılaştırıyor; sonra bu modelleri gerçek bare-metal donanıma indiriyor, gerçek bir MPU6050 ivmeölçeri çevrime sokuyor ve gecikmeyi, enerjiyi, Flash ile SRAM'i doğrudan ölçüyoruz. Ölçüm, soyut değerlendirmenin sessiz geçtiği birkaç etkeni açığa çıkarır — ve bunların bazıları, hangi modelin "en iyi" olduğu ya da sistemin gerçek-zamanlı çalışıp çalışmadığı gibi soruların cevabını değiştirir.

Katkılar

Bu ölçümlerin ortaya koyduğu başlıca sonuçlar şunlardır:

- **Koşullu fizibilite.** Çarpıcısı ve kayan-nokta birimi olmayan, sub-kilobayt bir mikrodenetleyicide gerçek-zamanlı (50 Hz) yinelemeli çıkarım fizibildir — ama koşulsuz değil; koşullarını karakterize ediyoruz.
- **İki bağımsız ön-koşul.** Gerçek-zaman, *hem* arama-tablosu (LUT) aktivasyonlarını *hem* yeterince hızlı (≥ 100 kHz) bir sensör veriyolunu gerektirir. Sensör-okuma yolu birinci derecede belirleyicidir ve çıkarım-only kıyaslamalara görünmezdir.

- **Rejim-bağımlı sıralama.** Evrensel olarak en iyi bir hücre yoktur: eşit gizli boyutta GRU önde; eşit *bayt*-bütçesinde FastGRNN'in yapısal sıkıştırması, sıralamayı verinin desteklediği yerde tersine çevirir (WISDM'de istatistiksel olarak anlamlı; HAPT'te beraberlik). Hangisinin "kazandığı", hangi kısıtın sabitlendiğine bağlıdır.
- **No-FPU mekanizması.** Derleyici optimizasyonunun neden neredeyse etkisiz kaldığının mekanistik açıklaması: FPU olmadığından her kayan-nokta işlemi önceden-derlenmiş bir soft-float çağrısıdır; fizibiliteyi araç zinciri değil, yazılım mimarisi belirler.
- **Ölçülmüş, çapraz-platform, dağıtım-farkında değerlendirme.** Gerçek donanımda, sensör döngü içindeyken, tahmin değil *ölçülmüş* gecikme/enerji/Flash/SRAM; ve tahmin düzeyinde üç mimari sınıfı boyunca (8-bit AVR, 16-bit MSP430, 32-bit ARM Cortex-M0+) doğrulanmış özdeşlik — burada bir donanım çarpıcısının, aktivasyon tablosuyla kabaca aynı gerçek-zaman payını geri kazandırdığını görüyoruz. Bu, önceki dördünü açığa çıkaran metodolojik araçtır.

Önceki sürümle ilişki

Bu çalışma, aynı arXiv girişindeki daha önceki bir reproduksiyonu gözden geçirir. O sürümün hücreler-arası karşılaştırma iddiası — bir hücrenin "en iyi" olduğu — hangi kısıtın sabitlendiğini belirtmediği için eksik tanımlıydı; burada aynı soruyu düzgün ölçer ve cevabın rejime bağlı olduğunu gösteririz. Bu bir geri çekme değil, bir arıtmadır; sürümler arası değişiklikler şeffaf biçimde belgelenmiştir.

Kod ve tekrarlanabilirlik / Yol haritası

Kod ve tekrarlanabilirlik. Tüm kaynak kod, eğitilmiş modeller, dışa aktarılmış Q15 ağırlıklar, deney başına günlükler ve dağıtım ikilileri, Apache 2.0 lisansı altında github.com/emre1998/fastgrnn-har adresinde açıktır. Tüm sonuçlar tek bir hesaplama ortamında (CPU) yeniden üretilir; ortam ve tohumlar raporlanmıştır.

Yol haritası. Bölüm 2, ilgili derlenmiş-RNN ve kenar-ML çalışmalarını değerlendirme rejimleri ekseninde derler. Bölüm 3 hücreleri ve low-rank/seyreklilik/Q15 sıkıştırma boru hattını biçimlendirir. Bölüm 4 deney düzenini tanımlar; Bölüm 5 eşit-kapasite ve eşit-bayt doğruluğundan bench dağıtımına, sensör-döngü ölçümüne ve çapraz-platform doğrulamasına uzanan sonuçları verir. Bölüm 6 başarısızlık kiplerini ayrı bir bölümde inceler; Bölüm 7 tartışır ve sınırları sıralar; Bölüm 8 sonuçlandırır.

2. İlgili Çalışma (onaylı TR)

Kompakt sinirsel modeller üzerine literatürü, konu başlıklarıyla değil, değerlendirme rejimiyle düzenliyoruz — çünkü bir modelin "yeterli" sayılıp sayılmadığı, hangi büyüklüğün sabitlendiğine ve neyin ölçüldüğüne bağlıdır. Her rejim meşru bir soruyu yanıtlar; ancak her biri, bir sonraki katmanın belirleyici olduğu etkenlere kördür.

2.1 Mimari kalite: doğruluk, kapasite sabitken. İlk rejim, modelleri sabit bir kapasitede (gizli boyut ya da parametre sayısı) doğrulukla karşılaştırır. Kompakt yinelemeli hücreler bu çizgide gelişti: LSTM [hochreiter1997lstm] ve GRU [cho2014gru] geçit mekanizmalarını kurdu; FastGRNN [kusupati2018fastgrnn], düşük-ranklı çarpanlara ayırma, seyreklik ve niceleme ile kilobayt-altı bir bütçede LSTM doğruluğuna yaklaştığını raporladı; Bonsai [kumar2017bonsai] ve ProtoNN [gupta2017protonn] aynı EdgeML programında ağaç- ve prototip-tabanlı alternatifler sundu [edgemi]. HAR alanında derin modeller yerleşiktir — DeepConvLSTM [ordonez2016deepconvlstm] ve iki kapsamlı derleme [wang2019harsurvey, demrozi2020harsurvey] bunu belgeler — ve standart kıyas kümeleri mevcuttur [anguita2013public, reyes2015transition]. Bu rejim "hangi mimari daha iyi öğrenir" sorusunu yanıtlar; dağıtım maliyetine kördür.

2.2 Sıkıştırma: parametre/bayt bütçesi sabitken. İkinci rejim, model boyutunu bir vekil olarak sabitler ve sıkıştırma yollarını karşılaştırır: budama [han2015pruning, frankle2019lt], niceleme [jacob2018integer, han2016deepcompression, hubara2017qnn], düşük-ranklı çarpanlara ayırma [sainath2013lowrank] ve iteratif sert eşikleme [blumensath2009iht]. Bu teknikler bir bayt bütçesinde doğruluğu korur; ancak gerçek bir mikrodenetleyicide bayt sayısı ne gecikmeyi ne enerjiyi tek başına belirler — ve hangi sıkıştırma yolunun "kazandığı", bütçeyi hangi ekseninde tuttuğunuza bağlıdır; bu bağımlılık nadiren birlikte incelenir.

2.3 Dağıtım: çıkarım-only, yetkin kenar donanımında. Üçüncü rejim modelleri gerçek donanıma indirir ve çıkarım gecikmesiyle belleği ölçer. Olgun bir araç zinciri bunu destekler: ARM Cortex-M için CMSIS-NN çekirdekleri [lai2018cmsisnn], TensorFlow Lite Micro [david2021tflitemicro], MCUNet [lin2020mcunet], karışık-hassasiyetli CMix-NN [capotondi2020cmixnn] ve MLPerf Tiny kıyası [banbury2021mlperf]; EdgeMark gibi sistemler bu araçları uçtan uca otomatikleştirip kıyaslar [hasanpour2025edgemark]. Görevimize en yakın çalışma, niceleme ve uyarlanabilir-derinlikli 1B evrişimli ağları düşük-güçlü RISC-V mikrodenetleyicilere indiren bir HAR dağıtımıdır [daghero2022har]. Ancak bu çalışmaların çoğu, kayan-nokta birimi ve donanım çarpıcısı bulunan Cortex-M/RISC-V sınıfı parçaları hedefler ve sensörden yalıtılmış çıkarımı ölçer. İki etken bu rejimin dışında kalır: çarpıcısız/FPU'suz tabanda aritmetik zeminin belirleyiciliği ve verinin çevrime nasıl girdiği.

2.4 Enerji ve fazlar: çoğunlukla tahmin, nadiren ölçüm. Dördüncü çizgi enerjiyi öne çıkarır; ancak sık atıf alan çalışmalar makro-ölçekli, eğitim-ağırlıklı ve tahmine dayalıdır [strubell2019energy, patterson2021carbon]. Yakın tarihli çalışmalar TinyML'de enerji ve gecikmeyi ölçülü ele almaya ve çıkarımı ön-çıkarma/çıkarma/son-çıkarma fazlarına ayırmaya başlar [bartoli2025benchmarking] — sensör-okuma yolunun ayrı ve birinci-derece olduğu yönündeki

bulgumuzla örtüşen bir sezgi. Yine de, çarpıcısız bir tabanda, gerçek sensör çevrim içindeyken akım sensörüyle ölçülmüş uçtan uca enerji görece seyrek.

2.5 Bu çalışmanın konumu. Bu dört rejim birlikte, bir modelin dağıtılabilişliğini üstten aşağıya doğru daraltan bir dizi soru oluşturur; ama en alttaki katman — çarpıcısız taban, sensör çevrimde ve enerji ölçülmüş — sistematik olarak nadiren ele alınır. Biz tam da orada duruyoruz: üç hücreyi (GRU, LSTM, FastGRNN) hem eşit-kapasite hem eşit-bayt rejiminde değerlendirip, ardından gerçek bare-metal donanımda, gerçek sensör döngüdeyken, ölçülmüş gecikme/enerji/bellekle ve üç mimari sınıfı boyunca sınıyoruz. Amacımız yeni bir sıkıştırma ya da hücre önermek değil; farklı rejimlerin farklı ve bazen çelişen cevaplar verdiğini gösterip, bunlardan hangisinin gerçek-zaman fizibilitesini gerçekten belirlediğini ölçmektir.

Eklenecek gerçek citation'lar (doğrulandı 31 Tem 2026)

- hasanpour2025edgemark — Hasanpour, Kirkegaard, Fafoutis. "EdgeMark: An Automation and Benchmarking System for Embedded Artificial Intelligence Tools." Journal of Systems Architecture 167:103488, 2025. (JSA venue sinyali)
- bartoli2025benchmarking — Bartoli, Veronesi, Giudici, Siorpaes, Trojaniello, Zappa. "Benchmarking Energy and Latency in TinyML: A Novel Method for Resource-Constrained AI." IJCNN 2025.
- daghero2022har — Daghero, Burrello, Xie, Castellano, Gandolfi, Calimera, Macii, Poncino, Jahier Pagliari. "Human Activity Recognition on Microcontrollers with Quantized and Adaptive Deep Neural Networks." ACM TECS 21(4):46, 2022.

3. Yöntem (onaylı TR)

Bu bölüm boyunca her seçimin yanına gerekçesini de yazıyoruz; çünkü çalışmanın tezi, kararların — hangi bütçe, hangi aktivasyon, hangi ölçüm ortamı — sonucu belirlediğidir. Gerekçesiz bir yöntem, bu makalede eksik bir yöntemdir.

3.1 Üç yinelemeli hücre. GRU [cho2014gru], LSTM [hochreiter1997lstm] ve FastGRNN [kusupati2018fastgrnn] hücrelerini karşılaştırıyoruz. Bu üçünü seçtik çünkü tasarım uzayını ölçülü biçimde tararlar: standart bir üç-kapılı hücre (GRU), durum-hücreli dört-kapılı bir hücre (LSTM) ve sıkıştırma-yerel bir hücre (FastGRNN). Üçü de aynı akış API'siyle (reset/step/predict), aynı sabit-nokta boru hattıyla ve aynı aktivasyon uygulamasıyla çalıştırılır; gerekçe: böylece kıyas, uygulama farklarını değil hücre yapısını yalıtır — hiçbir hücre kendi elverişli uygulamasıyla avantaj kazanmaz.

3.2 İki değerlendirme rejimi — yöntemin kalbi. Modelleri iki ayrı bütçede değerlendiriyoruz. **Eşit-kapasite:** gizli boyut sabit ($H=16$), doğruluk karşılaştırılır — bu "hangi mimari daha iyi öğrenir" sorusudur. **Eşit-bayt:** dağıtım ayak izi sabit (~ 283 sıfırdan-farklı parametre / ~ 566 B), her hücre kendi sıkıştırmasıyla o bütçeye indirilir — bu "hangisini dağıtmak daha iyi" sorusudur. Gerekçe: bunlar *farklı sorulardır* ve bir hücre birinde kazanıp diğerinde kaybedebilir. İkisini birbirine karıştırmak — bir bütçedeki üstünlüğü koşulsuz üstünlük sanmak — önceki sürümde düzelttiğimiz eksik-tanımın ta kendisidir. Rejimi açıkça belirtmek, "en iyi hücre" sorusunu ilk kez iyi-tanımlı hâle getirir.

3.3 Sıkıştırma boru hattı ve bütçe karşılaştırması. FastGRNN sıkıştırmayı yerel taşır: düşük-ranklı çarpanlara ayırma, IHT seyrekliği [blumensath2009iht] ve nicelenmiş ağırlıklar. GRU ve LSTM'i aynı bayt bütçesine iki yolla indiriyoruz — gizli boyutu küçültmek (shrink-H) *ya da* $H=16$ 'yı büyüklük-tabanlı budayıp seyrekleştirmek [han2015pruning] — ve her taban model için ikisinin daha iyisini alıyoruz.

Eşit-bayt rejiminde anlamlı soru, tüm hücreleri aynı biçimde sıkıştırmak değildir — dağıtımda böyle bir kısıt yoktur — her hücreyi elimizdeki en iyi yolla o bütçeye indirip hangisinin daha iyi dağıtıldığını sormaktır. Karşılaştırmayı üç noktada aynı zemine oturtuyoruz: taban modeller iki yolun daha iyisiyle temsil edilir; tüm hücreler eşit eğitim uzunluğunda (~ 200 dönem) koşulu — eğitim uzunluğu tek başına F1'i noktaya göre ~ 0.07 kaydırabildiğinden, bir dönem kazancının sıkıştırma kazancı gibi okunmaması için; ve bütçe her iki tarafta da sıfırdan-farklı ağırlık $\times 2$ B ile sayılır. Ortak son adım Q15 nicelemesidir [jacob2018integer] — FPU'suz donanımda bir seçim değil, zorunluluk; yapısal sıkıştırma ise her hücrede tasarım gereği farklıdır.

Bütçe *yaklaşık* eşittir, birebir değil: her hücrenin sıkıştırması hedefe yakın ayrık bir boyuta oturur, dolayısıyla dağıtılan ağırlık ayak izleri GRU 480 B, LSTM 472 B, FastGRNN 562 B'dir. Ölçülen sayıları olduğu gibi veriyoruz, tek bir nominal bütçeye yuvarlamadan; dikkat çekici olan, FastGRNN'in aslında en *büyük* ayak izine sahip olmasıdır — yani eşit-bayt kazanımları daha küçük bir modelden elde edilmiş değildir. Bayt sayımı ayrıca seyrek indeks depolamasını her iki tarafta da yok sayar: simetrik ama idealize bir cetvel; gerçek bir seyrek dağıtım indeks maliyeti öder (Sınırlar). Bu zeminde FastGRNN'in eşit-bayt üstünlüğü ölçülüdür — WISDM'de anlamlı, HAPT'te beraberlik.

3.4 MCU dağıtımı ve arama tablosu (LUT). Dağıtılan model, Python referansıla bit düzeyinde eşdeğerliği doğrulanmış tek bir taşınabilir C kaynağıdır; akış modunda örnek-örnek çalışır. Sigmoid ve tanh'ı $[-8, 8]$ aralığında 256-girişli bir arama tablosuyla hesaplıyoruz. Gerekçe: çarpıcısı ve FPU'su olmayan bir parçada her `expf / tanhf`, önceden-derlenmiş pahalı bir soft-float çağrısıdır; tabloya çevirince bu maliyet tek bir indeks-okumasına iner. Aşağıda göstereceğimiz gibi, bu LUT gerçek-zamanın bir konfor payı değil, çoğu hücre için bir *ön-koşuludur*.

3.5 Canlı-sensör protokolü. Fizibilitiyi, sensör çevrim dışıyken değil, gerçek bir MPU6050 ivmeölçer döngü içindeyken ölçüyoruz: 50 Hz'de I2C üzerinden okuma + normalize + çıkarım, uçtan uca zamanlanır; I2C hızı 10 kHz (muhafazakâr) ve 100 kHz (standart) olarak ayarlanır. Enerji,

INA226 akım sensörüyle sensörün de beslendiği raydan okunur; yani bildirilen güç, gerçek *sistem* gücüdür. Gerekçe: çalışmanın tezi fizibilitenin modelin değil, dağıtılmış sistemin bir özelliği olduğudur; sensör-okuma yolunu ölçümün dışında bırakmak, tam da belirleyici olabilecek etkeni gözden kaçırmak demektir. Enerjiyi **tahmin etmiyor, ölçüyoruz**; ve güç ile gecikmeyi ayrı ayrı ölçüp enerjiyi bu ikisinden türetiyoruz — hangi sayının ölçülmüş, hangisinin türetilmiş olduğunu metinde ayrı tutuyoruz.

3.6 Ölçüm ortamı — ve neden CPU. Tüm yazılım sonuçları tek bir ortamda, CPU üzerinde üretilmiştir (AMD Ryzen 7 6800H, tek iş parçacığına sabitlenmiş; Python/PyTorch sürümleri ve git düğümü kayıt altındadır). Gerekçe: deneylerin bir kısmı başlangıçta GPU'da koşmuştu ve GPU ile CPU sonuçları arasında küçük ama sistematik sapmalar gözledik — kaynağı, yinelemeli ağların cuDNN geri-yayılımındaki belirlenimsizlik ve TF32'nin kısaltılmış mantisidir; ayrıca çok-iş-parçacıklı BLAS indirgeme sırası da tekrar-üretilebilirliği bozuyordu. Bir makalede eğitim ortamının kendi içinde tutarlı olması gerekir; aksi halde "sonuçlarınız ölçtüğünüz cihaza bağlı" eleştirisi haklı olur. Bu yüzden GPU'da yapılmış deneyleri CPU'da yeniden ürettik ve **tek bir kayıt-ortamı** ilan ettik. Kullanılan işlemci modelini ve iş-parçacığı sayısını raporlamamızın nedeni de budur: tekrar-üretilebilirlik, ortamın tam olarak bildirilmesini gerektirir.

3.7 Çapraz-platform doğrulama. Aynı dağıtım kodunu üç mimari sınıfında çalıştırıyoruz: 8-bit AVR (Arduino), 16-bit MSP430 ve 32-bit ARM Cortex-M0+ (STM32G070). MSP430 ile G070'i **her ikisi de 16 MHz'de** kıyaslıyoruz; gerekçe: böylece kıyastaki tek değişken saat hızı değil, mimaridir (donanım çarpıcısının varlığı) — çarpıcının etkisi yalıtılmış olur. Eşdeğerliği tahmin (argmax) düzeyinde, her hücrenin kendi ana-bilgisayar referans çıktısına karşı doğruluyoruz; gerekçe: MSP430 ve G070 farklı soft-float kütüphaneleri kullandığından ara-değerler alt bitlerde ayrışabilir, ama sınıflandırma özdeş kalır — iddiamız "aynı tahminler", "bit-birebir gizli durum" değildir.

3.8 İstatistiksel işlem. Her sonucu beş tohum üzerinden, %95 yüzdellik bootstrap güven aralığıyla raporluyoruz; eşit-bayt üstünlüğünü, FastGRNN'in marjının en iyi taban yola göre güven aralığıyla değerlendiriyoruz — aralık sıfırı dışlıyorsa sonuç *desteklenmiş*, sıfırı kesiyorsa *beraberliktir*. Gerekçe: beş tohumla aralık geniş ve yaklaşıktır; ham tohum dağılımını birincil kayıt olarak sunuyor, üstünlüğü ancak aralığın gerçekten desteklediği güçte iddia ediyoruz. Bu, bir bütçedeki dar bir farkı süpürücü bir zafer gibi sunmayı engeller.

4. Deney Düzeni (onaylı TR)

4.1 Veri setleri. Üç HAR veri seti kullanıyoruz: HAPT [anguita2013public, reyes2015transition], WISDM ve PAMAP2. Gerekçe: tek bir veri seti, bir sıralamanın veri-setine mi özgü yoksa genel mi olduğunu gösteremez; rejim-bağımlılığı ancak birden çok sette görünür. Donanım deneyleri HAPT'te merkezlenir. Üç sette de girdi, 128-örneklik pencereler $\times$ 3 ivmeölçer eksenini olacak biçimde ortaklaştırılır. Gerekçe (yalın girdi): yalnızca ham ivmeölçer kullanmak, hedefimizin "en

küçük donanımda minimal ön-işleme" duruşuyla tutarlıdır — ama bu aynı zamanda PAMAP2'yi bir *sınır vakası* yapar: yalnızca üç eksenle (kalp-ritmi ve diğer IMU kanalları olmadan) hiçbir hücre orada kullanışlı doğruluğa ulaşamaz, bu yüzden PAMAP2 raporlanır ama sıralama iddiasına dahil edilmez (§5).

4.2 Ön-işleme ve bölme. Doğrulama, özne-farkında yapılır: eğitim öznelerinin son dördü doğrulamaya ayrılır. Gerekçe: aynı kişinin pencereleri hem eğitimde hem testte yer alırsa doğruluk yapay olarak şişer; özne-ayrımı bu sızıntıyı önler. Normalizasyon (kanal-başına ortalama/std) yalnızca eğitim kümesinden hesaplanır; gerekçe: test istatistiklerinin eğitime sızması için.

4.3 Eğitim protokolü. Adam ($lr = 10^{-3}$), çapraz-entropi kaybı, gradyan-kırpma (maks-norm 5.0), yığın boyutu 64 (eğitim) / 256 (doğrulama-test); en iyi doğrulama makro-F1'ine göre kontrol noktası seçilir. Eşit-kapasite rejimi 120 dönem, dağıtım-bütçesi rejimi 200 dönemdir (ve taban modeller de 200 dönemde yeniden koşular — §3.3). Her deney beş tohumla {0–4} tekrarlanır ve her tohum raporlanır, aykırı olan dahil. Ölçüt makro-F1'dir; gerekçe: sınıf dengesizliği altında makro-ortalama, azınlık sınıflarını çoğunluk kadar tartar.

4.4 Ön-kayıt (pre-registration). Karar kurallarını — hangi farkın "anlamlı" sayılacağını ve her sonucun hangi çerçeveyi tetikleyeceğini — herhangi bir sonuç görülmeden önce yazıp dondurduk. Gerekçe: bu, sonuç sonrası akıl yürütmeyi (post-hoc rationalization) engeller; sayılar üretildiğinde kendilerini yorumlar. Nitekim eşit-kapasite ön-kayıt, önceden taahhüt edilen yeniden-çerçeveleme kuralını tetikledi (§5.1): GRU, FastGRNN'i açık farkla geçti, naif üstünlük iddiası düştü ve makalenin çerçevesi tercih edilen bir anlatıdan değil, bu sonuçtan çıktı.

4.5 Güvenilirlik, birinci-sınıf eksen. Ortalamanın yanında standart sapmayı ve en-kötü-tohumu da birincil sonuç olarak raporluyoruz, dipnot olarak değil. Gerekçe: beş tohumdan birinde çöken bir hücre (bkz. FastGRNN düşük-rank kararsızlığı, §6), eşit ortalamada bile dağıtım için daha kötüdür; güvenilirlik, dağıtımda doğruluk kadar önemlidir.

4.6 Donanım düzeni. Birincil hedef MSP430G2553 (MSP-EXP430G2ET), 16 MHz, TI cl430 derleyicisi, -O3, `--use_hw_mpy=none` (donanım çarpıcısı kapalı — çarpıcısız-taban tezini teyit eder). Enerji, besleme rayına yerleştirilen bir INA226 akım sensörüyle ölçülür; sensör de aynı raydan beslenir. Canlı ölçümlerde MPU6050 ivmeölçer I2C üzerinden (10/100 kHz) çevrime alınır. Çapraz-platform doğrulama için aynı kod ayrıca Arduino Uno (8-bit AVR) ve STM32G070 (32-bit ARM Cortex-M0+) üzerinde koşular. Tüm dağıtım kodu, Python referansı ile bit düzeyinde eşdeğerliği doğrulanmış sabit-nokta C'dir.

5. Sonuçlar (onaylı TR — yazılıyor)

Sonuçları iki değerlendirme rejiminde (E1–E2), ardından mekanizma ve Pareto/nicelemede (E3–E5), sonra üç donanım katmanında (E6–E8) sunuyoruz. Her sonucu, aralığın gerçekten

desteklediği güçte iddia ediyoruz.

5.1 Eşit-kapasite: hangi mimari daha iyi öğrenir (E1)

Üç hücreyi de gizli boyut $H=16$ 'da, aynı protokolle eğitip makro-F1 karşılaştırıyoruz (beş tohum, %95 bootstrap GA):

Veri seti	GRU	LSTM	FastGRNN
HAPT	0.905 [0.876, 0.925]	0.884 [0.863, 0.906]	0.860 [0.838, 0.883]
WISDM	0.772 [0.762, 0.782]	0.746 [0.698, 0.789]	0.739 [0.717, 0.765]
PAMAP2*	0.327 [0.300, 0.367]	0.264 [0.246, 0.280]	0.290 [0.282, 0.299]

*PAMAP2 üç-eksen kısıtı altında sıralamaya dahil değildir (§4.1).

GRU, sıralanan iki veri setinde de (HAPT, WISDM) önde. Anlam: H sabit olduğundan bu, mimari kalitesinin — uygulama ya da bütçe değil — doğrudan bir ölçümüdür. Bu sonuç, önceki sürümden ödünç alınan "FastGRNN en doğru hücredir" önermesini çürütür; ön-kaydın Outcome 3'ünü tetikleyen ve makaleyi olgun çerçeveye çeviren nokta budur.

Ötesinde iki gözlem, seçimi kapasitenin tek başına yakalamadığını gösterir. LSTM en çok parametreye sahip olmasına rağmen hem en zayıf hem de en dengesiz hücredir (WISDM'de tohumlar arası 0.671–0.802 arasında salınır) — güvenilirlik ekseninin (§4.5) neden dipnot değil birincil sonuç olduğunu somutlar. FastGRNN ise en az parametreyle ($H=16$ 'da ~440, GRU'nun ~1110'una karşı) istikrarlı ve açık farkla ikinci gelir; kapasitede kaybeder ama *yoğunluk* olarak öne çıkar — bu, dağıtım-bütçesi sorusunda (§5.2) belirleyici olacak.

Önemli bir dürüstlük kaydı: bu makalenin bildirdiği sıralama tersinmesi burada, eşit kapasitede gerçekleşmez. Eşit kapasitede daha iyi hücre GRU'dur. Tersinme, farklı bir soruya — sabit *bayt* bütçesine — geçtiğimizde ortaya çıkar; onu §5.2'de ölçüyoruz.

5.2 Eşit-bayt: hangisini dağıtmak daha iyi (E2)

Aynı üç hücreyi bu kez sabit dağıtım bütçesine (~283 sıfırdan-farklı parametre / ~566 B) indiriyoruz; her hücre §3.3'teki en iyi yoluyla, bütçe her iki tarafta da eşit. FastGRNN'in üstünlüğünü, en iyi taban yola göre marjının %95 bootstrap aralığıyla değerlendiriyoruz — aralık sıfırı dışlıyorsa *desteklenmiş*, kesiyorsa *beraberlik*.

Veri seti	FastGRNN	En iyi taban (yol)	Marj (FG – taban) [%95 GA]	Sonuç
HAPT	0.869	0.901 GRU (budanmış)	–0.031 [–0.117, +0.028]	beraberlik (GA sıfırı kesiyor)

Veri seti	FastGRNN	En iyi taban (yol)	Marj (FG – taban) [%95 GA]	Sonuç
WISDM	0.800	0.732 GRU (budanmış)	+0.067 [+0.043, +0.091]	desteklenmiş
PAMAP2*	0.444	0.354 GRU (küçültülmüş)	+0.090 [+0.024, +0.155]	desteklenmiş (sıralama-dışı)

*PAMAP2 sıralama dışıdır (§4.1).

Sıralama, WISDM'de tersine döner — ve iddiamız tam olarak bu kadardır. WISDM'de FastGRNN en iyi tabanı +0.067 ile geçer; aralık sıfırı dışlar (desteklenmiş) ve sonuç sabit-nokta dağıtımında birebir üretilebilir. Buna karşılık HAPT'te tersinme gerçekleşmez: FastGRNN 0.869, GRU'nun budanmış 0.901'inin gerisindedir ve marjın aralığı sıfırı kestiği için bu bir beraberliktir — sessizce "FastGRNN 2/3 kazanıyor" demek, tam da düzeltmeye çalıştığımız türden bir aşırı-iddia olurdu.

Neden rejim cevabı değiştiriyor (mekanizma). Sabit baytta GRU, ya gizli boyutunu $H=16$ 'dan $\sim H=7$ 'ye düşürmek ya da ağırlıklarının büyük kısmını budamak zorundadır; her ikisi de kapasite kaybettirir. FastGRNN ise düşük-rank + IHT sayesinde $H=16$ 'yı bütçe içinde tutar. Yani eşit-kapasitede kaybeden hücre, sabit *bayt* altında yapısal sıkıştırmasıyla kapasiteyi koruyabildiği için öne geçebilir. Bu, "en iyi hücre" sorusunun neden tek bir cevabı olmadığıнын somut nedenidir: GRU daha iyi *hücre*dir (E1), FastGRNN daha iyi *sıkıştırma-kapasite ödünleşmesidir* (E2, desteklendiği yerde) — hangisinin kazandığı, hangi kısıtı sabitlediğinize bağlıdır.

Tabanların en iyi bütçe noktası çoğunlukla budamadan geldi (GRU'da hem HAPT hem WISDM); küçültme yalnızca PAMAP2'de öne çıktı — yol seçimini hücre başına en iyisiyle sabitlediğimizin (§3.3) somut kaydı, tam yol-yol matrisi Ek'te.

İki dürüstlük kaydı. Birincisi, marj *ölçülüdür*: tabanlar en iyi yollarını ve eşit dönemlerini aldığında görülen +0.067, yalnızca zayıf küçültme yolu raporlansaydı görünecek ~ 0.12 'lik farkın değil. İkincisi, FastGRNN'in HAPT bütçe dağılımı geniştir (bir tohum 0.708'e çöker); bu, düşük-rank kararsızlığının bir belirtisidir ve güvenilirlik açığını §6'da ayrı ele alıyoruz — ortalama yakinken bile beş tohumdan birinde çöken bir hücre, dağıtım için bir uyarıdır.

5.3 Mekanizma: derleyici optimizasyonu neden neredeyse etkisiz (E3)

Fizibiliteyi araç zincirinin mi yoksa yazılım mimarisinin mi belirlediğini ayırmak için tam bir optimizasyon taraması yaptık: 3 hücre $\times$ LUT{0,1} $\times$ -O{off,0,1,2,3,4} = 36 ölçüm. Yalnızca CCS optimizasyon seviyesi değişti; kod aynı kaldı.

Hücre	no-LUT: off $\rightarrow$ plato	LUT: off $\rightarrow$ plato
GRU	20.20 $\rightarrow$ 19.27 ms (~ -4.6)	13.04 $\rightarrow$ 12.12 ms (~ -7.1)
LSTM	22.80 $\rightarrow$ 22.80 ms (~ 0)	13.09 $\rightarrow$ 12.37 ms (~ -5.5)

Hücre	no-LUT: off → plato	LUT: off → plato
FastGRNN	27.2 → 26.1 ms (~%4)	15.3 → 13.9 ms (~%8.9)

Derleyici optimizasyonu neredeyse hiçbir şey kazandırmıyor (LUT'suz %0–4.6, LUT'lu %5–9) ve her durumda -O2'de doyuyor — -O3/-O4 fazladan bir şey vermiyor. Bir MCU dağıtımı için sezgiye aykırı bu sonucun bir mekanizması var, ve bu mekanizma makalenin kendi tezidir.

Kök neden — çarpıcısız/FPU'suz taban. MSP430G2553'te kayan-nokta birimi yoktur; dolayısıyla her kayan-nokta işlemi — çarpma-toplamalar da, `expf / tanhf` gibi transandantaller de — önceden-derlenmiş bir RTS *soft-float* kütüphane çağrısıdır. Projenin -O bayrağı bu kütüphane rutinlerinin içini değiştirmez; yalnızca etraflarındaki döngü/indeksleme "tutkalını" etkiler ki bu pay ihmal edilebilir. Yani MCU'yu yavaş yapan olgu (donanım çarpıcısı/FPU yokluğu) ile -O'yu etkisiz yapan olgu aynıdır — ikisi tek bir kök. `--use_hw_mpy=none` bunu teyit eder.

Neden önemli. Bu, "fizibiliteyi araç zinciri değil yazılım mimarisi belirler" katkısını bir Tartışma dipnotundan çıkarıp birinci-sınıf bir bulguya taşır: gerçek-zamana -O ile ulaşamazsınız; yazılımı değiştirmeniz gerekir. LUT'un neden bu kadar belirleyici olduğunun da açıklaması budur — LUT, transandantalleri soft-float çağrısından tek bir tablo-okumasına çevirir; bu, tam da -O'nun sizin için yapamayacağı şeydir. LUT'lu sütunun -O'dan biraz daha çok (%5–9) kazanması da tutarlıdır: tablolaşınca derlenebilir tutkal oranı artar, ama float çarpımlar hâlâ soft-float olduğu için kazanç sınırlı ve erken plato.

5.4 Boyut-doğruluk cephesi: bütçe seçilmiş mi, avantaj nereden (E4)

Eşit-bayt karşılaştırması (§5.2) tek bir bütçe noktasındadır (~566 B). Bunun keyfî seçilmiş bir nokta olup olmadığını ve FastGRNN'in avantajının tam olarak nereden geldiğini görmek için HAPT'te yoğun shrink-H cephesini süpürüyoruz: her hücre çeşitli gizli boyutlarda ($H \in \{4...12\}$), yoğun, 120 dönem, beş tohum (2 B/parametre, Q15). Her nokta kendi gerçek boyutunda çizilir.

Hücre (H)	Parametre	Makro-F1
GRU H4	138	0.787
GRU H6	240	0.881
GRU H8	366	0.899
GRU H10	516	0.916
FastGRNN H8	160	0.817
FastGRNN H12	284	0.817

Birincisi: bütçe seçilmiş değil. HAPT'te GRU cephesi FastGRNN'inkinin üzerindedir — bütçe civarında (240 parametrede GRU 0.881'e karşı 284 parametrede FastGRNN 0.817) ve daha büyük

bütçelerde açık ara. Yani §5.2'nin HAPT sonucu (GRU önde) 566 B'ye özgü bir artefakt değil; ölçülen tüm bütçelerde geçerli.

İkincisi — ve daha önemlisi: FastGRNN'in avantajının nereden geldiğini izole ediyor. Yalnızca H'yi küçültmek, FastGRNN'i HAPT'te GRU'dan daha bayt-verimli *yapmıyor*. Öyleyse FastGRNN'in WISDM ve PAMAP2'deki eşit-bayt kazanımları, hücrenin *özünde daha ucuz* olmasından değil, düşük-rank + seyrek yapı sayesinde H=16'yı bütçe içinde koruyabilmesinden gelir. Bu, §5.2'nin mekanizmasını keskinleştirir: tersinme, "küçük hücre" değil, "sabit baytta kapasite koruma" olgusudur.

Bir rigor kaydı (bilerek tek rejim). Cephedeki her nokta yoğun ve 120 dönemdir; §5.2'nin dağıtım-bütçesi noktalarını bu cepheye kasıtlı olarak bindirmiyoruz. Gerekçe: dağıtım noktaları daha uzun bir çizelge (200 dönem) kullanır ve HAPT'te LSTM'in bütçe noktası, aynı yoğun H=5 modelinin yalnızca fazladan dönemlerden +0.071 F1 kazanmış hâlidir (0.773 → 0.844). Bindirilseydi, bir *dönem* kazancı bir *sıkıştırma* kazancı gibi okunurdu — cephe bu tuzağı önlemek için ayrı tutulur.

5.5 Niceleme: Q15 doğruluğa mal oluyor mu (E5)

Q15 (16-bit sabit-nokta), FPU'suz donanımda zorunlu son adımdır (§3.3) — bir seçim değil. O hâlde kritik soru şudur: bu zorunlu adım doğruluktan ne götürür? Dağıtılan FP32 ve Q15 modellerini üç hücre × üç veri setinde karşılaştırıyoruz.

HAPT (dağıtım bütçesi noktası, örnek):

Hücre	FP32	Q15	$\Delta F1$
GRU	0.8715	0.8714	-0.0001
LSTM	0.7583	0.7583	~0
FastGRNN	0.8693	0.8695	+0.0002

Q15 neredeyse kayıpsızdır: dokuz hücre×veri-seti kombinasyonunun hepsinde $|\Delta F1| < 0.0002$. Niceleme, ölçülebilir düzeyde doğruluk götürmez.

Neden. Nicelemeden önce aktivasyon aralıklarını kalibre ediyoruz (§3.3); değerler Q15'in dinamik aralığında doymunluğa girmediği için kayıp sifıra yakın kalır. Bu, makalenin kendi katkılarından biridir — ama yeni bir *algoritma* değil, titizlikle uygulanan bir *reçete* olarak sunulur (§3, hedef-dışı listesi): Q15+kalibrasyon yeni bir niceleme yöntemi icat etmez, mevcut yöntemi ölçülebilir biçimde kayıpsız hâle getirir.

Anlamı. Q15 kayıpsız olduğu için dağıtılan model, eğitilen modeldir — sabit-noktaya geçmek için doğruluk feda edilmez. Bu, donanım katmanının (E6–E8) yazılım doğruluğunu miras aldığı, onu bozmadığını garanti eder. Ayrıca dağıtım C kodu, Python referansı ile bit düzeyinde eşdeğerdir; bu

daha güçlü belirlenimcilik iddiasını çapraz-platform bağlamında (E8) ayrı ele alıyoruz — orada "Q15↔FP32 sayısal uyumu" ile "platformlar-arası argmax özdeşliği" farklı iki ölçüdür.

5.6 Bench dağıtım: gerçek donanım, sensör yok (E6)

Yazılım doğruluğu Q15'te korunuyor (E5); şimdi bu modelleri MSP430'a indirip sensör bağlı değilken gecikme, güç, enerji ve belleği ölçüyoruz. Enerji için ayrımı baştan koyuyoruz: güç ölçülür, gecikme ölçülür, enerji bu ikisinden türetilir.

Hücre	Gecikme (LUT)	(no-LUT)	Güç	Enerji/pencere (LUT / no-LUT)	Flash	SRAM
GRU	12.12 ms ✓	19.27 ms	17.70 mW	27.5 / 43.7 mJ	5392 B	308 B
LSTM	12.37 ms ✓	22.10 ms ✗	17.70 mW	28.0 / 50.1 mJ	5742 B	324 B
FastGRNN	13.90 ms ✓	26.10 ms ✗	17.72 mW	31.5 / 58.9 mJ	5544 B	348 B

Bellek: üçü de sığar. En büyük Flash 5742 B (16 KB'nin ~%35'i), en büyük SRAM 348 B (512 B'nin ~%68'i) — dağıtılabilir. (Ağırlık-yalnız analitik ayak izi ayrı bir büyüklüktür: GRU 480 B, LSTM 472 B, FastGRNN 562 B; ikisini karıştırmıyoruz.)

Gecikme: LUT ile üçü de gerçek-zaman (12–14 ms < 20 ms); LUT olmadan LSTM ve FastGRNN düşer, GRU kıl payı geçer. LUT, hücreden bağımsız bir gerçek-zaman ön-koşuludur — bir FastGRNN hilesi değil.

Güç düz (~17.7 mW), hücreden ve LUT'tan bağımsız. Bir INA226 bit'i içinde eşit; yani güç platformun özelliğidir, modelin değil. Yalnızca güç sütununa bakmak, aktivasyon uygulamasının *bedava* olduğunu düşündürür — değildir: enerji/pencere düz değildir, çünkü güç farklı bir *gecikmeyle* çarpılır. LUT enerjiyi gücü değil, gecikmeyi kısararak düşürür (–%37 GRU, –%44 LSTM, –%46 FastGRNN).

Bir sınır kaydı: firmware örnekler arası uyku moduna girmez, meşgul-bekler; dolayısıyla enerji değerleri aktif rejimin üst sınırıdır (LPM'li düşürme gelecek iş, §7).

Ama dikkat — bu tablo yanıltıcı biçimde iyimser. Sensör devre dışı olduğundan LUT'lu her konfigürasyon "geçer" görünür. Sonraki katman (E7) tam da bunu bozar: gerçek sensörü çevrime soktuğumuzda tablonun büyük kısmı düşer. Bench, dağıtım fizibilitesinin bir *üst sınırıdır*, kendisi değil.

5.7 Sensör döngüde: fizibilitenin gerçek sınavı (E7)

Fizibilite iddiasını gerçek kılan hamle budur. Bench (E6) modeli bellekten besler; burada gerçek bir MPU6050 ivmeölçeri 50 Hz'de I2C üzerinden çevrime sokup sensör okuma + çıkarımı uçtan uca

zamanlıyoruz. Üç hücre $\times$ LUT{0,1} $\times$ I2C{10, 100 kHz}, 512 ardışık örnek ortalaması, 20 ms bütçesine karşı:

Hücre	LUT @100 kHz	LUT @10 kHz	no-LUT @100 kHz	no-LUT @10 kHz
GRU	12.01 ✓	20.43 ✗	20.03 ✗	27.48 ✗
LSTM	12.98 ✓	20.71 ✗	22.49 ✗	30.12 ✗
FastGRNN	13.98 ✓	22.26 ✗	27.45 ✗	34.98 ✗

Dört sütundan yalnızca biri geçer. Ve bu tek sütun, iki bağımsız ön-koşulun aynı anda sağlandığı sütundur: LUT-tabanlı aktivasyonlar ve hızlı (100 kHz) sensör veriyolu.

Bulgu 1 — Gerçek-zaman yalnızca LUT + 100 kHz ile mümkün. Bu sütunda GRU (12.01) < LSTM (12.98) < FastGRNN (13.98), üçü de bütçe içinde. Başka hiçbir kombinasyon geçmez.

Bulgu 2 — İki bağımsız ön-koşul. LUT tek başına yetmez: LUT'lu ama 10 kHz'de üçü de düşer. Hızlı veriyolu tek başına yetmez: 100 kHz'de ama LUT'suz üçü de düşer. Fizibilite, ikisinin birden sağlanmasına bağlıdır — çıkarımın kendisi kadar, onu besleyen yol da belirleyicidir.

Bulgu 3 — Sensör-okuma yolu birinci derecededir ve çıkarım-only kıyaslamalara görünmezdir. 10 kHz'de, herhangi bir çıkarım başlamadan önce yalnızca sensör okuması 8.4 ms tutar; bu tek başına, en hızlı LUT'lu konfigürasyonu bile bütçe dışına iter. 100 kHz'de aynı okuma ~0.8 ms'dir. Modeli bellekten besleyen bir kıyas (E6) bu 8.4 ms'yi hiç görmez — bench'te "geçer" görünen konfigürasyonlar, gerçek edinim yolu eklendiğinde düşer. Makalenin çekirdek gözlemi tam da budur: fizibiliteyi belirleyen bir etken, standart değerlendirmenin kör noktasında durur.

Bulgu 4 — no-LUT hiçbir veriyolu hızında gerçek-zaman olamaz. LUT'suz çıkarım tek başına ≥ 19 ms olduğundan (E6), üzerine herhangi bir sensör maliyeti eklenince bütçe kesin aşılır.

Enerji — gerçek sistem gücü. INA226 ile ölçülen sistem gücü (MCU + I2C + MPU6050) hücreden ve LUT'tan bağımsızdır: 100 kHz'de ~32.0 mW, 10 kHz'de ~33.4–34.4 mW. Bu, yalnız-MCU bench değerinin (17.7 mW) kabaca iki katıdır; sensörü bağlamak sistem gücünü ikiye katlar ve pil ömrünü yarıya indirir. Dürüst gerçek-dünya dağıtım enerjisi budur, bench değeri değil.

Neden bu sayılara güveniyoruz. Her canlı gecikme ölçümü, bağımsız bench (sıfır-girdi) sayısıyla ~%5 içinde uyuştur (ör. GRU 12.01 $\approx$ 12.12; FastGRNN 13.98 $\approx$ 13.90; no-LUT LSTM 22.5 $\approx$ 22.1). Bu, hem canlı düzeneği hem de daha önceki bench sayılarını çapraz-doğrular.

Bu katman, makalenin harita–arazi ayrımını somutlar: bench *haritası* dört konfigürasyondan üçünü "uygun" gösterir; sensörlü *arazi* yalnızca birinin geçtiğini söyler. İki ön-koşulun birlikte gerekliliği — ve sensör yolunun görünmezliği — ancak araziyi ölçünce ortaya çıkar.

5.8 Çapraz-platform: üç mimari sınıfı (E8)

Şimdiye kadarki dağıtım MSP430 üzerindedir. Bu son katman, aynı modellerin farklı mimari sınıflarına *taşınabilirliğini* doğrular — bu, makalenin omurgası değil, bir yan-doğrulama: fizibilite tezi çarpıcısız MSP430 üzerinde durur; burada gösterdiğimiz, aynı işin daha yukarı ve daha aşağı silikonda da geçerli olduğudur. Değişmeyen dağıtım kodunu üç sınıfta koşuyoruz: 8-bit AVR (Arduino, v1'den), 16-bit MSP430 ve 32-bit ARM Cortex-M0+ (STM32G070).

Özdeşlik. Beş gömülü test penceresinde, her hücrenin G070 tahmini kendi ana-bilgisayar referansı (host-C `C_PRED`) her iki LUT knob'unda da 5/5 örtüşür; MSP430 da aynı referansa uyduğundan, geçişli olarak $G070 == \text{host-C} == \text{MSP430}$ tahmin (argmax) düzeyinde. İddia düzeyi bilerek "aynı tahminler"dir, "bit-birebir gizli durum" değil: MSP430 (TI soft-float) ve G070 (GCC soft-float) farklı kütüphaneler kullandığından ara-değerler alt bitlerde ayrışabilir, ama sınıflandırma özdeş kalır (§3.7).

Gecikme (saf çıkarım, her ikisi de 16 MHz — tek değişken mimari):

Hücre	MSP430 LUT / no-LUT	G070 LUT / no-LUT
GRU	12.12 / 19.27 ms (kıl payı)	7.24 / 12.02 ms
LSTM	12.37 / 22.10 ✗	7.11 / 13.50 ms
FastGRNN	13.90 / 26.10 ✗	10.84 / 17.00 ms

Bulgu 1 — Donanım çarpıcısı her no-LUT konfigürasyonunu gerçek-zamana sokar. Çarpıcısız MSP430'da LUT'suz GRU kıl payında, LSTM ve FastGRNN düşer; aynı byte-birebir modeller çarpıcılı G070'te LUT'suz bile 20 ms'yi geçer (%60 / %68 / %85). Matristeki en kötü vaka — FastGRNN no-LUT — G070'te bile 17 ms'de geçer.

Bulgu 2 — Çarpıcı $\approx$ LUT kaldırıcı. G070'te LUT'suz GRU (12.02 ms), MSP430'da LUT'lu GRU'ya (12.12 ms) %0.8 içinde eşittir. İki farklı kaldırıcı — bir donanım çarpıcısı ile bir aktivasyon tablosu — aynı saatte kabaca aynı gerçek-zaman payını geri kazandırır.

Anlamı — omurga tezini güçlendirir. En zor vaka (çarpıcısız MSP430) çalışıyorsa, yaygın bir M0+ aynı modeli daha rahat çalıştırır. Bu yüzden çarpıcısız taban bir *stres* örneğidir: fizibiliteyi belirleyen etkenler (LUT, aritmetik zemin) orada en büyük ve en görünürdür; daha yetkin donanımda etkileri yumuşar. Üç mimari sınıfı boyunca aynı dağıtımın tahmin-özdeş çalışması, çalışmanın taşınabilirlik iddiasını somutlar.

6. Başarısızlık Çözümlemesi (onaylı TR)

Bir fizibilite iddiasının değeri, sınırlarının ne kadar keskin çizildiğiyle ölçülür. Bu bölüm, sistemin nerede ve neden kırıldığını tek yerde toplar — bir dürüstlük jesti olarak değil, yöntemin parçası olarak: her başarısızlık kipi, "gerçek-zamanlı HAR çarpıcısız bir MCU'da fizibildir" iddiasının tam olarak hangi koşullarda geçerli olduğunu keskinleştirir.

6.1 Düşük-rank kararsızlığı — güvenilirlik kırılması. FastGRNN'in eşit-bayt dağılımı geniştir; HAPT'te beş tohumdan biri 0.71'e çöker (§5.2) — ortalamayı yakın tutan ama güvenilirliği bozan bir sıçrama. Sıkıştırma boru hattını adım adım ayırıştıran bir ablasyon (yoğun → düşük-rank → +seyreklik → +Q15), kararsızlığın kaynağını düşük-rank adımına yerleştirir; IHT/seyreklik ya da Q15'e değil. Bu, HAPT'e özgü, tohuma duyarlı bir olgudur. Sonuç dağıtım için önemlidir: beş tohumdan birinde çöken bir hücre, eşit ortalamada bile riskli bir dağıtım adaydır — güvenilirliği bu yüzden birincil eksen olarak raporluyoruz (§4.5).

6.2 Fizibilitenin çifte sınırı. Gerçek-zaman iki bağımsız ön-koşula bağlıdır (§5.7) ve her biri ayrı bir başarısızlık kipidir. LUT kaldırıldığında çıkarım tek başına ≥ 19 ms olur (no-FPU soft-float, §5.3) ve hiçbir veriyolu hızı bunu kurtaramaz. Sensör veriyolu 10 kHz'e düştüğünde okuma tek başına 8.4 ms tutar ve en hızlı LUT'lu konfigürasyonu bile bütçe dışına iter. Fizibilite ancak ikisi de sağlandığında vardır; bu, kademeli bir bozulma değil, keskin bir "ya hep ya hiç" sınırıdır.

6.3 PAMAP2 — sınır vakası. Yalnızca üç eksenle (kalp-ritmi ve diğer IMU kanalları olmadan) hiçbir hücre PAMAP2'de kullanışlı doğruluğa ulaşamaz: sınıf-başına F1'lerin çoğu 0.25'in altındadır, bir sınıf hiç öğrenilmez ve tekrarlanan aynı koşullar tek tek sınıf skorlarını 0.2–0.5 oynatır. Bu bir *model* başarısızlığı değil, bir *girdi* başarısızlığıdır — seçtiğimiz yalın (üç-eksen) girdi, PAMAP2'nin ayırt ediciliği için yetersizdir. Bir sensör eklemek onu kurtarabilirdi, ama bu minimal-ön-işleme duruşundan ödün olurdu; bu yüzden PAMAP2 raporlanır, sıralanmaz (§4.1).

6.4 Warm-up payı — bir tasarım kısıtı. Yinelemeli gizli durum, pencere başında sıfırlandıktan sonra kararlı bir tahmine oturmak için zaman ister: 100 pencere üzerinde medyan 74, en kötü 125 örnek. Firmware her pencerede durumu sıfırlar ve sınıflandırmayı pencere sonunda (128. örnek) yapar; en kötü warm-up (125) < 128 olduğundan sınıflandırma her zaman oturmuş bir durum üzerindedir — v1'in "1.48 s boyunca yanlış çıktı" okuması bu yüzden hatalıdır, çünkü ara çıktı hiç okunmaz. Ne var ki pay yalnızca 3 örnektir (128 – 125): pencere uzunluğu en kötü warm-up tarafından *alttan sınırlanır*. Bu ek bir gecikme değil, pencere uzunluğuna binen bir tasarım kısıtıdır ve daha kısa pencerelerin en kötü durumda oturmayacağını söyler.

6.5 Zayıf sınıf. Sınıf-başına çözümleme, hataların düzgün dağılmadığını gösterir: merdiven-inme (DOWNSTAIRS) tutarlı biçimde en zayıf sınıftır — literatürde bilinen-zor bir sınıf. Bunu kovalamıyoruz; kovalamak, ek özellik/sensör gerektirmeden çalışan yalın-ön-işleme satış noktasını zayıflatırdı. Sınırı beyan ediyoruz.

7. Tartışma ve Sınırlar (onaylı TR)

7.1 Tartışma

Merkezi bulgu: fizibilite bir dağıtım-mühendisliği problemidir. Aynı üç model, aynı donanım — ama bench (E6) dört konfigürasyondan üçünü "uygun" gösterirken, gerçek sensörlü ölçüm (E7)

yalnızca birinin geçtiğini söyler. Aradaki fark, modelin bir özelliği değildir; onu çevreleyen mühendisliğin — aktivasyonun nasıl hesaplandığı ve verinin nasıl geldiği — özelliğidir. Fizibiliteyi belirleyen bu etkenler, çıkarım-only değerlendirmenin kör noktasında durur: bir kıyas modeli bellekten beslediğinde, 8.4 ms'lik sensör okuması hiç görünmez. Çalışmanın çekirdeği budur — soyut değerlendirme (harita), fiziksel ölçümün (arazi) ortaya çıkardığı belirleyici etkenleri saklar.

"En iyi hücre" rejime bağlıdır. Eşit kapasitede GRU daha iyi öğrenir (E1); sabit baytta FastGRNN, kapasiteyi düşük-rank+seyrek yapıyla koruyabildiği için desteklenen yerde (WISDM) öne geçer (E2, E4). İkisi çelişmez; farklı soruları yanıtlarlar. Evrensel olarak en iyi bir hücre yoktur, ve "hangisi kazanır" ancak hangi kısıtın sabitlendiği söylendiğinde iyi-tanımlıdır. Bu, tek bir hücreyi taçlandırmaktan bilinçli bir geri adımdır: katkı bir kazanan değil, sorunun kendisinin yeniden çerçevelenmesidir.

Aritmetik zemin, ortak iplik. Tekrar eden mekanizma tek bir olgudur: FPU/çarpıcı yokluğu. Bu olgu hem MCU'yu yavaşlatır, hem derleyici optimizasyonunu etkisiz kılar (E3, çünkü her float işlem önceden-derlenmiş soft-float'tır), hem de LUT'u zorunlu yapar (transandantalleri soft-float çağrısından tablo-okumasına çevirdiği için). Çapraz-platform katmanı (E8) aynı ipliği tersinden gösterir: bir donanım çarpıcısı eklendiğinde, LUT'un çarpıcısız tabanda kazandığı pay kabaca geri gelir. Fizibiliteyi araç zinciri değil, yazılım mimarisinin aritmetik zeminle etkileşimi belirler.

Kenar-ML değerlendirmesi için anlamı. Bu sonuçların hiçbiri, dağıtımın önemli olduğunu "keşfetmez" — bu bilinir. Katkımız, çıkarım-only değerlendirme ile gerçek dağıtım arasındaki boşluğun, en zor donanımda ve sensör çevrimdeyken *ne kadar büyük ve belirleyici* olduğunu ölçmektir. Eğer bu boşluk sıralamaları tersine çevirebiliyor ve fizibiliteyi görünmez ön-kosullara bağlayabiliyorsa, kenar-ML dağıtım kararları çıkarım-only kanıtla verildiğinde eksik bilgiyle verilir. Ölçülmüş, sensör-döngülü, dağıtım-dürüst değerlendirme bir lüks değil, doğru kararın koşuludur.

7.2 Sınırlar

- **Donanım deneyleri HAPT-merkezlidir;** üç-veri-seti taraması yazılımdır. Donanım fizibilite sonucu yapısal olarak veri-bağımsızdır (hesap maliyeti veriden bağımsızdır) — sessizce genellemiyor, böyle beyan ediyoruz.
- **Enerji ve sensör çözümlemesi MSP430'a özgüdür.** Çapraz-platform katmanı (E8) tahmin-düzeyi özdeşliği ve gecikmeyi kapsar, enerjiyi değil; başka platformlarda sistem enerjisi ayrıca ölçülmelidir.
- **Firmware örnekler arası uyku moduna girmez** (meşgul-bekler); bildirilen enerji aktif-rejimin *üst sınırıdır*. LPM0/3 ile görev-döngülü düşürme gelecek iştir ve enerjiyi yaklaşık aktif-oran kadar azaltır.
- **Sensör ve çıkarım gecikmesinin ayrımı, 1 ms'lik zamanlayıcı altında çözülemez;** uçtan-uca değer güvenilirdir, ayrım değil.

- **Dağıtılan firmware tek bir konfigürasyondur** (HAPT, tohum 0); bunu beş-tohum dağılımının yerine değil, yanında raporluyoruz.
- **I2C yalnızca 10 ve 100 kHz'de sınılandı**; hızlı-mod (400 kHz) test edilmedi.
- **Eğitim uzunluğu, deney aileleri arasında bir confound'dur** (§3.3); cepheler bu yüzden bilerek üst üste bindirilmez.
- **Bayt-muhasebesi seyrek indeks depolamasını yok sayar** (her iki tarafta simetrik ama idealize, §5.2); gerçek bir seyrek dağıtım indeks maliyeti öder.
- **Sıkıştırma yapısı hücre-başına tasarım gereği farklıdır** (adalet için, §3.3); ortak son adım Q15'tir, "hepsi aynı biçimde sıkıştırıldı" değildir.

8. Sonuç (onaylı TR)

Çarpıcısı ve kayan-nokta birimi olmayan, sub-kilobayt bir mikrodenetleyicinin gerçek-zamanlı insan aktivite tanıma yapıp yapamayacağını sorduk ve iddiayla değil ölçümle yanıtladık. Yanıt: yapabilir — ama koşulsuz değil. Gerçek-zaman, iki bağımsız mühendislik ön-kosuluna bağlıdır: arama-tablosu aktivasyonları ve yeterince hızlı bir sensör veriyolu. İkisinden biri eksikse fizibilite kırılır — ve bu belirleyici etkenlerden biri, sensör-okuma yolu, çıkarım-only kıyaslamalara görünmezdir.

Hangi hücrenin "en iyi" olduğu tek bir cevap taşımaz: eşit kapasitede GRU daha iyi öğrenir, sabit baytta FastGRNN'in yapısal sıkıştırması desteklenen yerde öne geçer. O yüzden katkımız kazanan bir hücre değil; sorunun rejime bağlı olduğunu göstermek ve bunu mümkün kılan ölçülmüş dağıtım çerçevesini kurmaktır. Tekrar eden mekanizma tek bir olguya iner — FPU/çarpıcı yokluğu — ki bu hem derleyici optimizasyonunu etkisiz kılar hem de arama-tablosunu zorunlu yapar; bir donanım çarpıcısı eklendiğinde aynı pay geri gelir.

Çalışmanın kahramanı bir algoritma değil, bir ispattır: yeterli dağıtım mühendisliğiyle, gerçek-zamanlı yinelemeli algılama üretimdeki en küçük silikonların bir kısmına ulaşır — ve tam olarak ne zaman ulaşmadığı karakterize edilmiştir. Yeni bir gezegen değil; metodolojik ve ölçülmüş bir mühendislik sonucu.

8.1 Önceki Sürümle İlişki

Bu çalışma, aynı arXiv girişindeki daha önceki bir sürümü gözden geçirir; bir geri çekme değil, bir arıttır. Değişenler şeffaf biçimde şöyledir:

- **Hücre-üstünlüğü iddiası düzeltildi.** Önceki sürümün ödünç aldığı "FastGRNN en doğru hücredir" iddiası, hangi kısıtın sabitlendiğini belirtmediği için eksik-tanımlıydı; burada ölçülür ve rejime bağlı hâle getirilir (eşit-kapasite GRU, eşit-bayt FastGRNN — desteklendiği yerde).

- **Kıyaslanamaz sayılar emekli edildi.** Önceki sürümdeki 54 s / 30.5× / %96.7 figürleri, sensörlü erken bir canlı-mod deneyinden gelen, saf-hesaplama latency'siyle kıyaslanamaz değerlerdi; ana tablolardan çıkarıldı ve yerlerini -O-duyarsızlığı mekanizması ile ölçülmüş enerji aldı.
- **Sensör-döngü katmanı eklendi.** İki ön-koşul ve sensör-yolunun görünmezliği, ancak gerçek sensör çevrimdeyken ortaya çıkar; bu katman yeni.
- **Üçüncü mimari sınıfı eklendi.** Çapraz-platform doğrulama artık 8-bit AVR ve 16-bit MSP430'a ek olarak 32-bit ARM Cortex-M0+'ı da kapsar.
- **İstatistiksel titizlik.** Bootstrap güven aralıkları ve dürüst-dar iddialar (WISDM'de desteklenmiş, HAPT'te beraberlik); tek ve tekrar-üretilebilir bir CPU ortamı.

Önceki sürümün geçerli katkıları — çapraz-platform bit-eşdeğerlik, LUT reçetesi, INA226 ölçülmüş enerji, warm-up çözümlemesi — bu sürümde de durur. Değişen, tek bir ödünç iddiadır; çerçeve tercih edilen bir anlatıdan değil, ölçülen sonuçtan çıkar.

Durum: TÜRKÇE TASLAK MÜHÜRLENDİ (2 Ağu 2026)

Sekiz bölüm tam: Abstract · Giriş · İlgili Çalışma · Yöntem · Deney Düzeni · Sonuçlar (E1–E8) · Başarısızlık Çözümlemesi · Tartışma+Sınırlar · Sonuç+Önceki-Sürüm.

Kalan (sonraki faz)

- [] İngilizce çeviri → paper/en/sections/*.tex (mevcut v1, ÜZERİNE)
- [] Format / **JSA uyumluluğu** (elsarticle sınıfı, Highlights, data statement, generative-AI beyanı) — HENÜZ YAPILMADI
- [] 3 yeni citation'ı references.bib'e ekle (hasanpour2025edgemark, bartoli2025benchmarking, daghero2022har)
- [] Ek: yol-yol sıkıştırma matrisi (shrink vs pruned tam tablo)
- [] Figürler (F1–F7 zaten hazır) yerleşimi